Image Processing
Assignment 1

Timothy van der Valk
2336715

September 28, 2026

Task 1 — Edge Detection

I have implemented an edge detection pipeline consisting of the following steps:

1. Apply a Gaussian or Median filter. For Gaussians, we used $\sigma = (k-1)/5$ as recommended by the book [1, p. 114], where k is the kernel size.
2. Apply the edge magnitude filter using Sobel kernels. The magnitudes were normalized based on the largest value, because otherwise the image would get too dark and not be visible after thresholding.
3. Apply thresholding from pixel intensity 75. This value has been chosen because it resulted in the most performant and consistent results for all kernel sizes.

Analysis for Low-Noise Images

The results for a clean tram image can be seen in Table 1. The Median filter performs best on the smaller kernel, where it can remove small structures that are irrelevant to edge detection. As the kernel size grows, the Median filter quickly flattens many important surfaces, leaving only a few main edges visible after thresholding. The Gaussian in comparison performs far better but becomes thicker due to our increasing value for σ.

Analysis for High-Noise Images

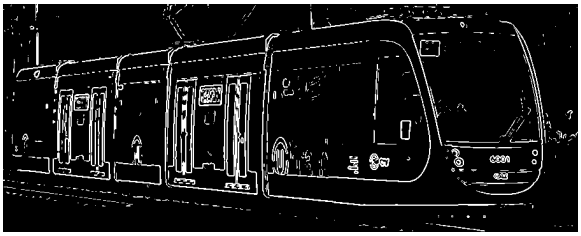
The results for a noisy tram image can be seen in Table 2. In this case, it is clear that the Gaussian kernel is far worse for smaller kernels. This is caused by its inability to remove strong, small noise effectively when the noise is allowed to dominate a small amount of samples. The Median filter performs as expected by efficiently removing small structures of noise. However, it suffers from the same problem as before, where it flattens surfaces to the same brightness. This causes a majority of the edges to be lost.



Reference image.



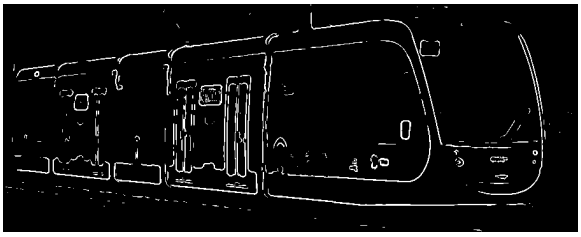
Gaussian 3x3 with $\sigma = 0.4$.



Median 3x3.



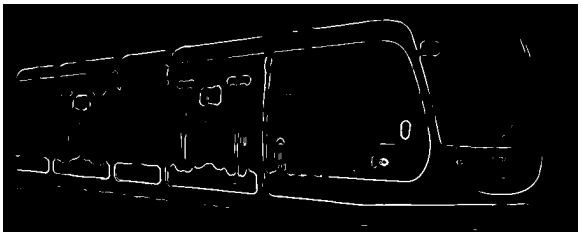
Gaussian 7x7 with $\sigma = 1.2$.



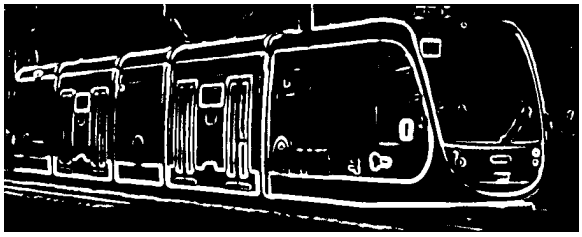
Median 7x7.



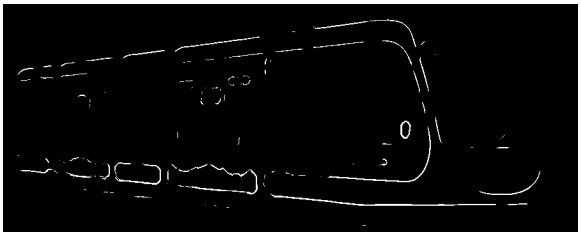
Gaussian 11x11 with $\sigma = 2$.



Median 11x11.



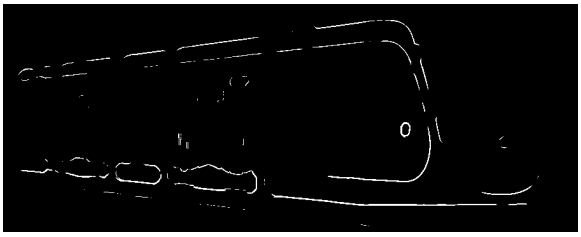
Gaussian 15x15 with $\sigma = 2.8$.



Median 15x15.



Gaussian 19x19 with $\sigma = 3.6$.

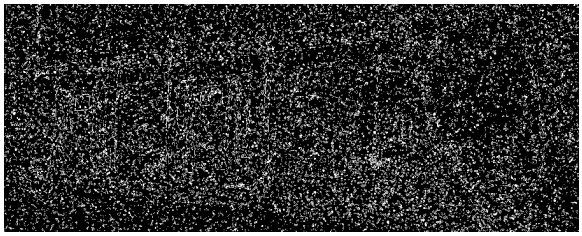


Median 19x19.

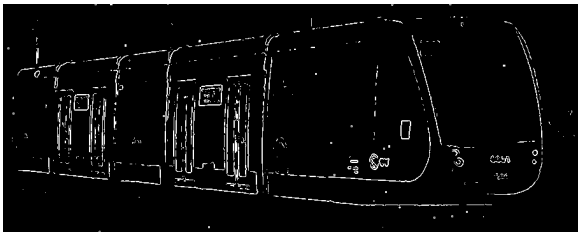
Table 1: Edge detection pipeline applied to clean tram images.



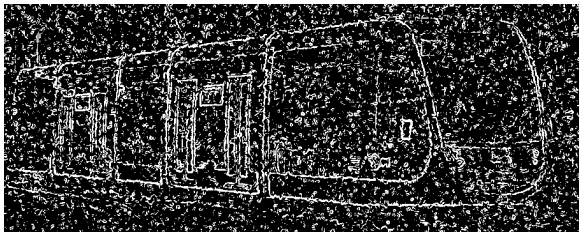
Reference image.



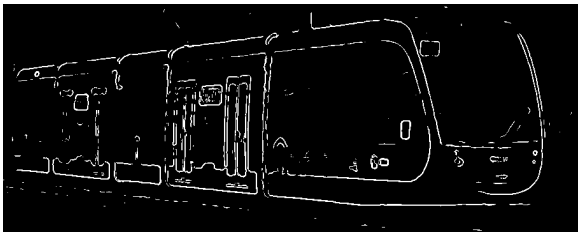
Gaussian 3x3 with $\sigma = 0.4$.



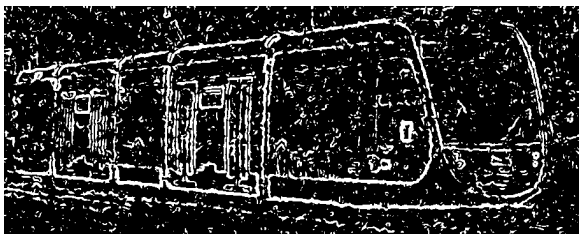
Median 3x3.



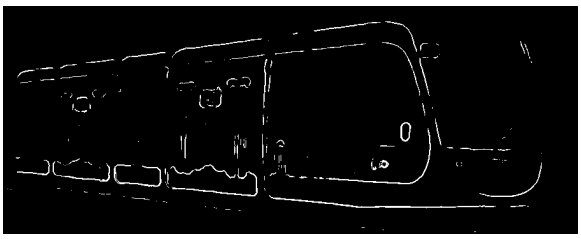
Gaussian 7x7 with $\sigma = 1.2$.



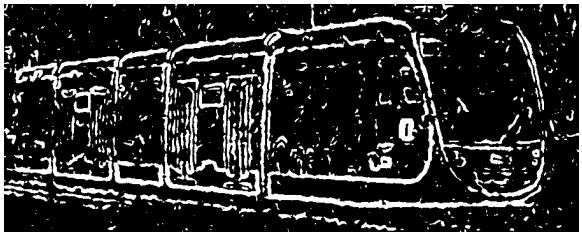
Median 7x7.



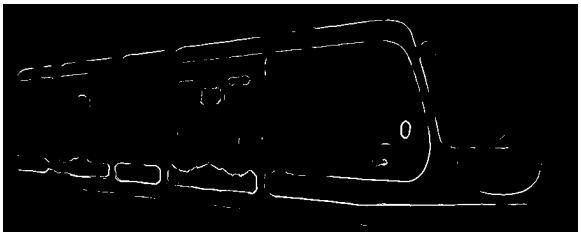
Gaussian 11x11 with $\sigma = 2$.



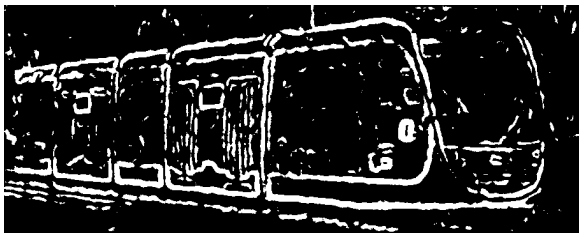
Median 11x11.



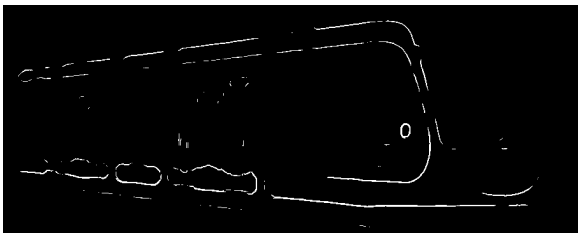
Gaussian 15x15 with $\sigma = 2.8$.



Median 15x15.



Gaussian 19x19 with $\sigma = 3.6$.



Median 19x19.

Table 2: Edge detection pipeline applied to noisy tram images.

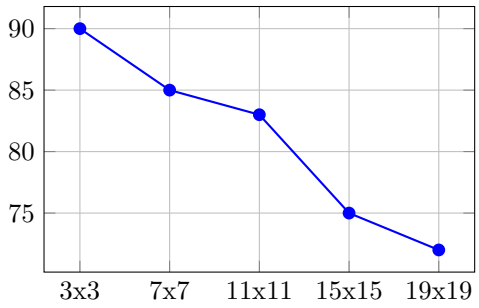
Task 2 — Grayscale Erosion

I applied Grayscale Erosion to the tram images with increasing kernel sizes. The square structuring elements had a constant value of 0.

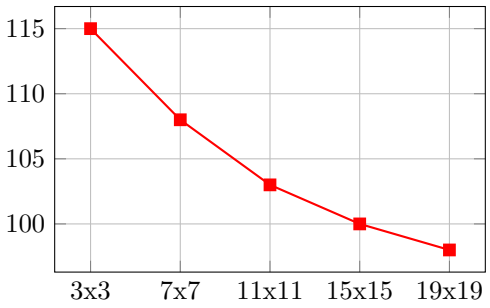
Analysis

The effect of Grayscale Erosion with a zero kernel comes down to taking the minimum of the neighborhood around a pixel. This can be observed by the fact that there are square patterns forming around pixels of lowest intensity. See Table 3.

It can be expected that as the minimization-kernel increases in size, the number of unique pixel intensities drop as they become the same as the local minimum. This in turn also causes a decrease in average intensity. These trends can be seen in Figure 1.



Intensity count by structure element size.



Average intensity by structure element size.

Figure 1: Plot of the number of unique intensities (left) and the average pixel intensity (right) of Grayscale Erosion applied with increasing structure element sizes.



Reference Image.



Grayscale Erosion with a flat 3x3 element.



Grayscale Erosion with a flat 7x7 element.



Grayscale Erosion with a flat 11x11 element.



Grayscale Erosion with a flat 15x15 element.



Grayscale Erosion with a flat 19x19 element.

Table 3: Grayscale Erosion pipeline applied to tram images.

Task 3 — Binary Closing

I have implemented Binary Closing with white pixels representing foreground. The structure elements are squares with all values set to true.

Analysis

The effect of Binary Closing is to first perform a Dilation, followed by an Erosion. This causes holes and gaps between white pixels to close up if they are smaller than twice the structure size. See Table 4. This effect can be seen in the 3x3 image, for example, where the black dot in the top left is closed up by the surrounding white pixels. For the larger structure sizes, we see that gaps between large blocks are closed up for the 7x7 element, and that the large black square on the bottom right is eliminated in 11x11.

Therefore, we would expect that the number of white foreground pixels increases when we use larger structure sizes. This effect can be seen in Figure 2.

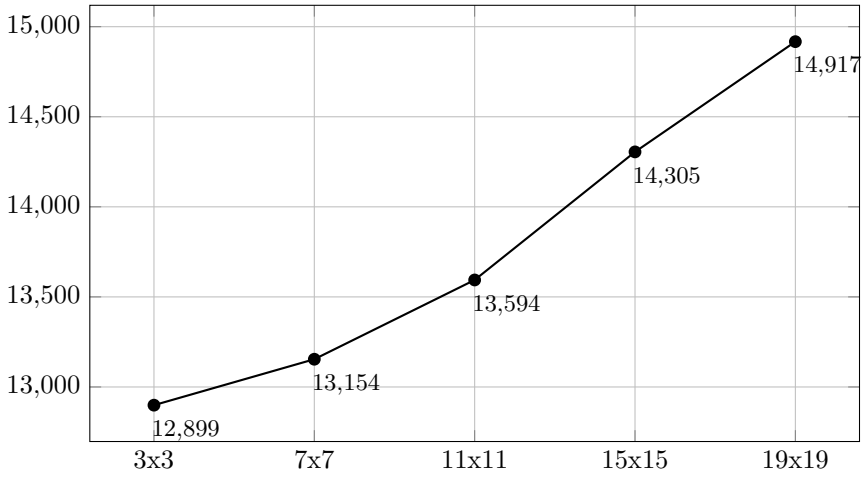
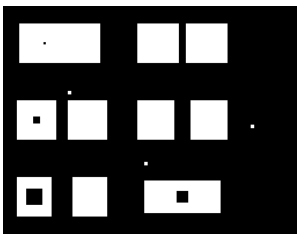
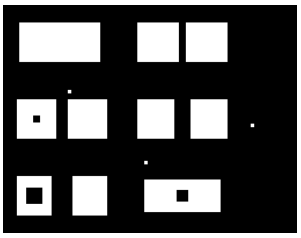


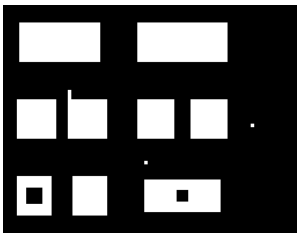
Figure 2: Plot of the number of foreground pixels after applying Binary Closing.



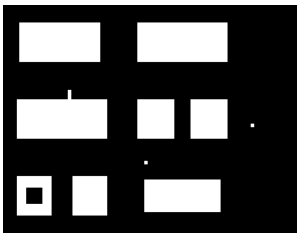
Reference Image.



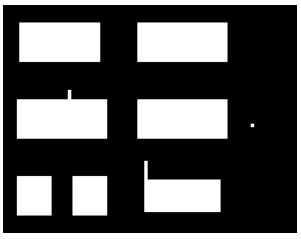
Binary Closing with a flat 3x3 element.



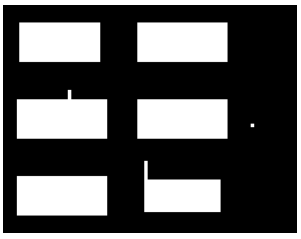
Binary Closing with a flat 7x7 element.



Binary Closing with a flat 11x11 element.



Binary Closing with a flat 15x15 element.



Binary Closing with a flat 19x19 element.

Table 4: Binary Closing pipeline applied to test images.

Extra — Canny Edge Detection

I implemented Canny Edge Detection with the following steps based on [2]:

1. Smooth the image using a Gaussian filter. I generally used $\sigma = 1$, unless the image has a higher noise.
2. Compute the edge magnitude using Sobel filters.
3. Perform edge localization by testing for local maxima.
4. Perform edge tracing and hysteresis thresholding on the local maxima. The thresholds are a percentage of the maximum edge magnitude.
5. Color the pixels in the same hysteresis flood group with the same random color. This gives an insight into the efficiency of finding longer edge segments, and is useful for debugging.

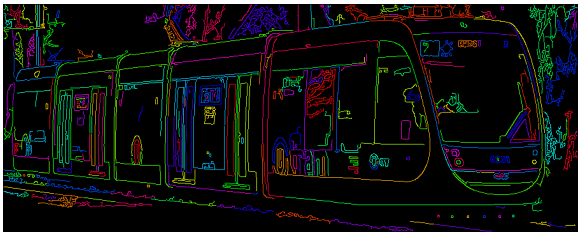
Depending on the presence of noise and other weak backgrounds edges in the image, I tweaked the Gaussian smoothing and high hysteresis threshold.

Analysis

See the results in Figure 5. It is clear that compared to thresholding-based edge detection, the Canny Edge Detector is much better adapted for detail and precise edge extraction. The pixel groupings represented by the colors are connected sections of edges, and could be further processed for the purpose of recognizing shapes.
For the fourth image in Figure 5, the Canny Edge Detector almost perfectly found the edges of the airplane and the high-frequency edges on the exhaust. In the last image with cables, I had to use a higher smoothing and hysteresis threshold to cut away the small edges corresponding to the grass in the background.
My analysis for the effect of the two thresholds, is that the low threshold is far less influencial, and only in certain cases introduces a slightly longer group of pixels if it is lowered. The high threshold is a key parameter to control the minimum edge magnitude relative to the maximum, when an edge is started with flood fill. I increased this parameter for images 1 and 5 due to their strong background edges. This cleared away some of them. For image 2, I reduced the threshold to obtain more edges, in particular for the hair.



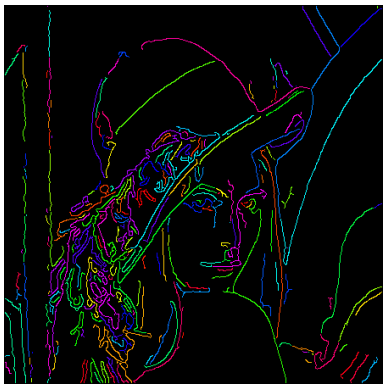
Reference image.



Canny Edge Detection with $\sigma = 1$ and $h = 0.3$.



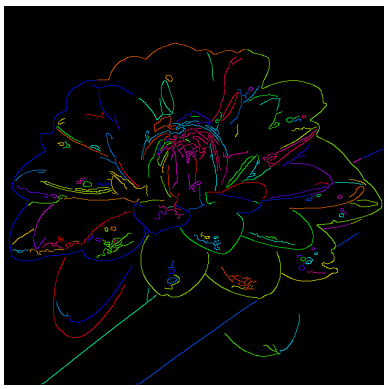
Reference image.



Canny Edge Detection with $\sigma = 1$ and $h = 0.15$.



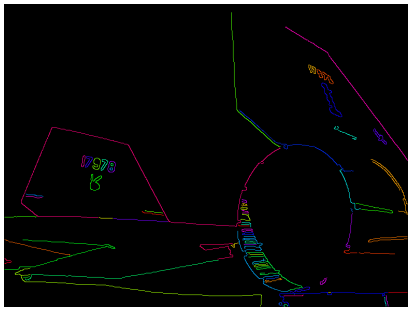
Reference image.



Canny Edge Detection with $\sigma = 1$ and $h = 0.2$.



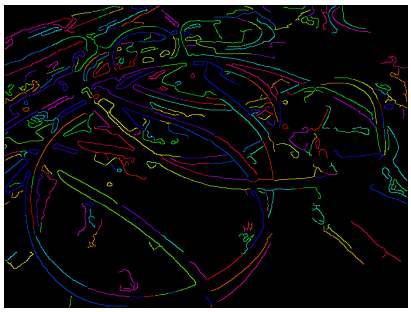
Reference image.



Canny edge Detection with $\sigma = 1$ and $h = 0.2$.



Reference image.



Canny Edge Detection with $\sigma = 2$ and $h = 0.3$

Table 5: Canny Edge Detection applied to a variety of images.

References

[1] Principles of Digital Image Processing: Fundamental Techniques, Wilhelm Burger, Mark J. Burge. Springer 2009.

[2] Principles of Digital Image Processing: Advanced Methods, Wilhelm Burger, Mark J. Burge. Springer 2013.